Laporan Tugas Kecil 2

IF2211 - Strategi Algoritma

Voxelization Objek 3D menggunakan Octree

Disusun oleh:

Wisa Ahmaduta Dinutama/18223003

Nazwan Siddqi Muttaqin/18223066

Program Studi Teknik Informatika

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

2025

A. Penjelasan Algoritma

a. Konsep Utama *Divide and Conquer*

Algoritma yang diimplementasikan adalah Pembangunan Octree Rekursif. Algoritma ini secara alami mengimplementasikan strategi *divide and conquer* karena setiap langkah rekursifnya memecah masalah (ruang tiga dimensi dan kumpulan *faces*) menjadi delapan upamasalah yang lebih kecil dan independen antara satu sama lain.

i. Gambaran Umum

Pada setiap level rekursi, algoritma melakukan tiga fase *divide and conquer*

No.	Nama	Deskripsi
1	***Divide***	Membagi *bounding box* dari kubus saat ini menjadi 8 upakubus yang sama besar dengan membelah tiap sumbu di titik tengahnya
2	***Conquer***	Untuk setiap upakubus, saring *faces* yang berpotongan dengan *bounding box* upakubus tersebut, lalu bangun octree pada upakubus tersebut secara rekursif
3	***Combine***	Kumpulkan seluruh *leaf node* yang memiliki *face* sebagai *voxel*

ii. Prosedur

1. **Kalkulasikan ruang awal *Object* sebagai suatu kubus**

   Mekanismenya adalah dengan mencari *Vertex* terkecil dan terbesar dari *Object* untuk merepresentasikan kubus dalam bentuk *Bounding Box*

2. **Bagi Kubus awal menjadi 8 upakubus**

   Cara kerjanya adalah dengan menghitung titik tengah dari kubus pada ketiga sumbu. Kemudian, digunakan indeks upakubus 0-7 yang direpresentasikan dalam bilangan biner 3-bit, di mana setiap bit akan menentukan posisi upakubus terhadap titik tengah pada masing-masing sumbu kubus *parent*. Proses ini akan menghasilkan 8 upakubus yang tidak tumpang tindih

3. **Saring *Faces* pada kubus tersebut dan tentukan *Faces* apa saja berpotongan dengan setiap upakubus hasil pembagian**

   Separating Axis Theorem (SAT) digunakan untuk menguji persimpangan yang menentukan *faces* mana saja yang akan berada di dalam upakubus hasil pembagian

4. **Ulangi proses 2 dan 3 secara rekursif (pembagian secara terus menerus untuk setiap upakubus yang dihasilkan)**

   Adapun *base case* (kondisi stop membagi) dari algoritmanya adalah sebagai berikut

   a. Tidak ada *faces* di dalam kubus

      Kubus akan ditandai *skipped* (untuk keperluan statistik) dan dikembalikan

   b. Kedalaman nol (*leaf*)

      simpan semua *faces* yang ada ke dalam kubus ini dan kembalikan. Kubus ini adalah voxel yang terisi

5. **Setelah rekursi selesai, kumpulan *Voxel* (kubus tanpa upakubus dengan *depth* 0) akan dikumpulkan kembali secara rekursif**

   Voxel-voxel inilah yang nantinya akan merepresentasikan kubus-kubus kecil yang menggambarkan objek tiga dimensi tersebut

b. Implementasi *Concurrency* (Bonus)

Untuk mengoptimasi waktu eksekusi, program menerapkan *concurrency* pada fase *Divide* di algoritma *Octree*. Program memproses pembelahan 8 anak ruang (oktan) secara serentak (paralel) menggunakan fitur bawaan Golang, yaitu Goroutine dan sync.WaitGroup.

Adapun alur kerjanya adalah sebagai berikut:

1. **Goroutine (go func):** Setiap perhitungan oktan baru dipisah ke dalam *thread* yang berbeda agar berjalan bersamaan.
2. **Sinkronisasi (sync.WaitGroup):** Program menahan eksekusi fungsi utama menggunakan perintah wg.Wait(). Perintah ini memastikan program tidak melangkah ke tahap selanjutnya sebelum kedelapan goroutine melaporkan bahwa tugas mereka telah selesai (wg.Done()).
3. **Penggabungan (*Combine*):** Setelah semua proses paralel selesai, hasil dari masing-masing oktan dimasukkan ke dalam *array* node.children .

Pendekatan ini terbukti secara signifikan memangkas waktu komputasi *voxelization*, terutama pada model 3D berskala besar.

B. *Source Code*

Komponen	Kode

Package dan import	`package main`
	`import (`
	`  "bufio"`
	`  "fmt"`
	`  "math"`
	`  "os"`
	`  "path/filepath"`
	`  "strconv"`
	`  "strings"`
	`  "sync"`
	`  "time"`
	`)`

Struct Definition	`type Vertex struct {`
	`  x, y, z float64`
	`}`
	
	`type Face struct {`
	`  a, b, c Vertex`
	`}`
	
	`type BoundingBox struct {`
	`  min, max Vertex`
	`}`
	
	`type OctreeNode struct {`
	`  depth int`
	`  children [8]*OctreeNode`
	`  faces []Face`
	`  boundingBox BoundingBox`
	`  skipped bool`
	`}`

Fungsi Menghitung *Bounding Box Root*	`func ComputeRootBound(faces [] Face) BoundingBox {`
	`  minX := faces[0].a.x`
	`  minY := faces[0].a.y`
	`  minZ := faces[0].a.z`
	`  maxX := faces[0].a.x`
	`  maxY := faces[0].a.y`
	`  maxZ := faces[0].a.z`
	`  for _, t := range faces {`
	`  for _, v := range [] Vertex{t.a, t.b, t.c} {`

```
      if v.x < minX { minX = v.x }
      if v.y < minY { minY = v.y }
      if v.z < minZ { minZ = v.z }

      if v.x > maxX { maxX = v.x }
      if v.y > maxY { maxY = v.y }
      if v.z > maxZ { maxZ = v.z }
    }
  }
  sizeX := maxX - minX
  sizeY := maxY - minY
  sizeZ := maxZ - minZ
  maxSize := math.Max(math.Max(sizeX, sizeY), sizeZ)

  return BoundingBox{
    min: Vertex{x: minX, y: minY, z: minZ},
    max: Vertex{x: minX + maxSize, y: minY + maxSize, z: minZ +
maxSize},
  }
}
```

Fungsi Menghitung *Bounding Box Child*

```
func ComputeChildBound (parent BoundingBox, i int) BoundingBox {
  min := parent.min
  max := parent.max

  midX := (min.x + max.x) / 2
  midY := (min.y + max.y) / 2
  midZ := (min.z + max.z) / 2

  var childMin, childMax Vertex

  if i&1 == 0 {
    childMin.x = min.x
    childMax.x = midX
  } else {
    childMin.x = midX
    childMax.x = max.x
  }

  if i&2 == 0 {
    childMin.y = min.y
    childMax.y = midY
  } else {
    childMin.y = midY
    childMax.y = max.y
  }
```

	``
	  if i&4 == 0 {
	  childMin.z = min.z
	  childMax.z = midZ
	  } else {
	  childMin.z = midZ
	  childMax.z = max.z
	  }
	
	  return BoundingBox{min: childMin, max: childMax}
	}
	``

Fungsi Mengecek *Face* yang Berpotongan dengan *Bounding Box*	```
	func faceIntersectsBox(face Face, box BoundingBox) bool {
	  // move triangle so box center at origin
	  cx := (box.min.x + box.max.x) * 0.5
	  cy := (box.min.y + box.max.y) * 0.5
	  cz := (box.min.z + box.max.z) * 0.5
	
	  hx := (box.max.x - box.min.x) * 0.5
	  hy := (box.max.y - box.min.y) * 0.5
	  hz := (box.max.z - box.min.z) * 0.5
	
	  v0x := face.a.x - cx
	  v0y := face.a.y - cy
	  v0z := face.a.z - cz
	
	  v1x := face.b.x - cx
	  v1y := face.b.y - cy
	  v1z := face.b.z - cz
	
	  v2x := face.c.x - cx
	  v2y := face.c.y - cy
	  v2z := face.c.z - cz
	
	  // compute edges
	  e0x := v1x - v0x
	  e0y := v1y - v0y
	  e0z := v1z - v0z
	
	  e1x := v2x - v1x
	  e1y := v2y - v1y
	  e1z := v2z - v1z
	
	  e2x := v0x - v2x
	  e2y := v0y - v2y
	  e2z := v0z - v2z
	``

```
// 1 - test axes L = edge cross axes
// e0
p0 := v0z*e0y - v0y*e0z
p2 := v2z*e0y - v2y*e0z
minP, maxP := p0, p2
if minP > maxP { minP, maxP = maxP, minP }
rad := hz*math.Abs(e0y) + hy*math.Abs(e0z)
if minP > rad || maxP < -rad { return false }

p0 = v0x*e0z - v0z*e0x
p2 = v2x*e0z - v2z*e0x
minP, maxP = p0, p2
if minP > maxP { minP, maxP = maxP, minP }
rad = hx*math.Abs(e0z) + hz*math.Abs(e0x)
if minP > rad || maxP < -rad { return false }

p1 := v1y*e0x - v1x*e0y
p2 = v2y*e0x - v2x*e0y
minP, maxP = p1, p2
if minP > maxP { minP, maxP = maxP, minP }
rad = hx*math.Abs(e0y) + hy*math.Abs(e0x)
if minP > rad || maxP < -rad { return false }

// e1
p0 = v0z*e1y - v0y*e1z
p2 = v2z*e1y - v2y*e1z
minP, maxP = p0, p2
if minP > maxP { minP, maxP = maxP, minP }
rad = hz*math.Abs(e1y) + hy*math.Abs(e1z)
if minP > rad || maxP < -rad { return false }

p0 = v0x*e1z - v0z*e1x
p2 = v2x*e1z - v2z*e1x
minP, maxP = p0, p2
if minP > maxP { minP, maxP = maxP, minP }
rad = hx*math.Abs(e1z) + hz*math.Abs(e1x)
if minP > rad || maxP < -rad { return false }

p0 = v0y*e1x - v0x*e1y
p1 = v1y*e1x - v1x*e1y
minP, maxP = p0, p1
if minP > maxP { minP, maxP = maxP, minP }
rad = hx*math.Abs(e1y) + hy*math.Abs(e1x)
if minP > rad || maxP < -rad { return false }
```

```
// e2
p0 = v0z*e2y - v0y*e2z
p1 = v1z*e2y - v1y*e2z
minP, maxP = p0, p1
if minP > maxP { minP, maxP = maxP, minP }
rad = hz*math.Abs(e2y) + hy*math.Abs(e2z)
if minP > rad || maxP < -rad { return false }

p0 = v0x*e2z - v0z*e2x
p1 = v1x*e2z - v1z*e2x
minP, maxP = p0, p1
if minP > maxP { minP, maxP = maxP, minP }
rad = hx*math.Abs(e2z) + hz*math.Abs(e2x)
if minP > rad || maxP < -rad { return false }

p1 = v1y*e2x - v1x*e2y
p2 = v2y*e2x - v2x*e2y
minP, maxP = p1, p2
if minP > maxP { minP, maxP = maxP, minP }
rad = hx*math.Abs(e2y) + hy*math.Abs(e2x)
if minP > rad || maxP < -rad { return false }

// 2 - test bounding box axes
minX, maxX := v0x, v0x
if v1x < minX { minX = v1x }
if v1x > maxX { maxX = v1x }
if v2x < minX { minX = v2x }
if v2x > maxX { maxX = v2x }
if minX > hx || maxX < -hx { return false }

minY, maxY := v0y, v0y
if v1y < minY { minY = v1y }
if v1y > maxY { maxY = v1y }
if v2y < minY { minY = v2y }
if v2y > maxY { maxY = v2y }
if minY > hy || maxY < -hy { return false }

minZ, maxZ := v0z, v0z
if v1z < minZ { minZ = v1z }
if v1z > maxZ { maxZ = v1z }
if v2z < minZ { minZ = v2z }
if v2z > maxZ { maxZ = v2z }
if minZ > hz || maxZ < -hz { return false }
```

	``go
	  // 3 - triangle plane vs box
	  nx := e0y*e1z - e0z*e1y
	  ny := e0z*e1x - e0x*e1z
	  nz := e0x*e1y - e0y*e1x
	
	  d := -(nx*v0x + ny*v0y + nz*v0z)
	
	  r := hx*math.Abs(nx) + hy*math.Abs(ny) + hz*math.Abs(nz)
	  s := d
	
	  if s > r || s < -r {
	  return false
	  }
	
	  return true
	}
	``

Fungsi untuk Menyaring *List of Faces* yang akan Berada di *Bounding Box Child Node*	```go
	func FilterFaces(faces []Face, box BoundingBox) []Face {
	  filtered := make([]Face, 0, len(faces))
	
	  for _, face := range faces {
	  if faceIntersectsBox(face, box) {
	  filtered = append(filtered, face)
	  }
	  }
	
	  return filtered
	}
	``

Fungsi untuk *Parsing Object* Menjadi *List of Faces*	```go
	func ParseObj(filename string) ([]Face, error) {
	  file, err := os.Open(filename)
	  if err != nil {
	  return nil, fmt.Errorf("gagal membuka file: %w", err)
	  }
	  defer file.Close()
	
	  var vertices []Vertex
	  var faces []Face
	  scanner := bufio.NewScanner(file)
	
	  for scanner.Scan() {
	  line := strings.TrimSpace(scanner.Text())
	  if line == "" || strings.HasPrefix(line, "#") {
	  continue
	  }
	``

	``go
	parts := strings.Fields(line)
	switch parts[0] {
	case "v":
	if len(parts) >= 4 {
	x, _ := strconv.ParseFloat(parts[1], 64)
	y, _ := strconv.ParseFloat(parts[2], 64)
	z, _ := strconv.ParseFloat(parts[3], 64)
	vertices = append(vertices, Vertex{x: x, y: y, z: z})
	}
	case "f":
	if len(parts) >= 4 {
	var indices []int
	for i := 1; i < len(parts); i++ {
	vals := strings.Split(parts[i], "/")
	idx, _ := strconv.Atoi(vals[0])
	indices = append(indices, idx-1)
	}
	for i := 1; i+1 < len(indices); i++ {
	if indices[0] < len(vertices) && indices[i] <
	len(vertices) && indices[i+1] < len(vertices) {
	faces = append(faces, Face{
	a: vertices[indices[0]],
	b: vertices[indices[i]],
	c: vertices[indices[i+1]],
	})
	}
	}
	}
	}
	}
	return faces, scanner.Err()
	}
	``
	Fungsi untuk Mengonversi *List of Bounding Boxes* Menjadi *Object*
		``go
		func ExportToObj(boxes []BoundingBox, outputPath string) error {
		file, err := os.Create(outputPath)
		if err != nil {
		return err
		}
		defer file.Close()
		
		writer := bufio.NewWriter(file)
		vertexOffset := 1
		
		for _, box := range boxes {
		min, max := box.min, box.max
		vertices := []Vertex{
		``
		Fungsi untuk Mengonversi *List of Bounding Boxes* Menjadi *Object*

	``
	{min.x, min.y, min.z}, {max.x, min.y, min.z}, {max.x, max.y, min.z}, {min.x, max.y, min.z},
	{min.x, min.y, max.z}, {max.x, min.y, max.z}, {max.x, max.y, max.z}, {min.x, max.y, max.z},
	}
	
	for _, v := range vertices {
	fmt.Fprintf(writer, "v %.6f %.6f %.6f\n", v.x, v.y, v.z)
	}
	
	facesIdx := [][]int{
	{1, 3, 2}, {1, 4, 3}, {5, 6, 7}, {5, 7, 8},
	{1, 2, 6}, {1, 6, 5}, {4, 8, 7}, {4, 7, 3},
	{1, 5, 8}, {1, 8, 4}, {2, 3, 7}, {2, 7, 6},
	}
	
	for _, f := range facesIdx {
	fmt.Fprintf(writer, "f %d %d %d\n", f[0]-1+vertexOffset, f[1]-1+vertexOffset, f[2]-1+vertexOffset)
	}
	vertexOffset += 8
	}
	
	return writer.Flush()
	}
	``

Fungsi untuk Membangun *Octree*	```
	func BuildOctree(faces []Face, bound BoundingBox, depth int) *OctreeNode {
	node := &OctreeNode{
	depth: depth,
	boundingBox: bound,
	}
	
	if len(faces) == 0 {
	node.skipped = true
	return node
	}
	
	if depth == 0 {
	node.faces = faces
	return node
	}
	
	var wg sync.WaitGroup
	for i := 0; i < 8; i++ {
	wg.Add(1)
	``

	``go
	    go func(childIdx int) {
	      defer wg.Done()
	      childBound := ComputeChildBound(node.boundingBox, childIdx)
	      filteredFaces := FilterFaces(faces, childBound)
	      node.children[childIdx] = BuildOctree(filteredFaces, childBound, depth-1)
	    }(i)
	  }
	  wg.Wait()
	
	  return node
	}
	``

Fungsi untuk Mengoleksi *Voxel* dari *Leaves Octree*	```go
	func CollectVoxelBoxes(node *OctreeNode, boxes *[]BoundingBox) {
	  if node == nil || node.skipped {
	    return
	  }
	  if node.depth == 0 {
	    if len(node.faces) > 0 {
	      *boxes = append(*boxes, node.boundingBox)
	    }
	    return
	  }
	  for i := 0; i < 8; i++ {
	    CollectVoxelBoxes(node.children[i], boxes)
	  }
	}
	``

Fungsi untuk Mendapatkan Statistik *Octree*	```go
	func CollectStats(node *OctreeNode, maxDepth int, created []int, skipped []int) {
	  if node == nil {
	    return
	  }
	  level := maxDepth - node.depth
	  created[level]++
	
	  if node.skipped {
	    skipped[level]++
	    return
	  }
	  for i := 0; i < 8; i++ {
	    if node.children[i] != nil {
	      CollectStats(node.children[i], maxDepth, created, skipped)
	    }
	``

	``go
	  }
	}
	``
	``go
	func PrintOctree(node *OctreeNode) {
	  var printNode func(node *OctreeNode, depth int)
	  printNode = func(node *OctreeNode, depth int) {
	    if node == nil {
	      fmt.Printf("%snil\n", strings.Repeat("│ ", depth))
	      return
	    }
	    prefix := strings.Repeat("│ ", depth)
	    fmt.Printf("%sNode(depth=%d, faces=%d)\n", prefix, node.depth, len(node.faces))
	    fmt.Printf("%sBoundingBox(min=(%.2f, %.2f, %.2f), max=(%.2f, %.2f, %.2f))\n",
	      prefix, node.boundingBox.min.x, node.boundingBox.min.y, node.boundingBox.min.z,
	      node.boundingBox.max.x, node.boundingBox.max.y, node.boundingBox.max.z)
	    for i, child := range node.children {
	      if child != nil {
	        fmt.Printf("%s├── Child %d:\n", prefix, i)
	        printNode(child, depth+1)
	      }
	    }
	  }
	  printNode(node, 0)
	}
	``
	Fungsi untuk Memvisualisasikan *Octree*

	``go
	func main() {
	  if len(os.Args) != 3 {
	    fmt.Println("Cara penggunaan: go run main.go <path_file.obj> <depth_maksimum>")
	    return
	  }
	
	  filename := os.Args[1]
	  depth, err := strconv.Atoi(os.Args[2])
	  if err != nil || depth < 0 {
	    fmt.Println("Error: Parameter kedalaman (depth) harus berupa bilangan bulat positif.")
	    return
	  }
	
	  fmt.Printf("Memproses file: %s dengan kedalaman maksimal: %d\n", filename, depth)
	``
	Program Utama

```go
  faces, err := ParseObj(filename)
  if err != nil {
    fmt.Printf("Error validasi input: %v\n", err)
    return
  }
  if len(faces) == 0 {
    fmt.Println("Error: Tidak ada data bidang (faces) yang valid
di file OBJ.")
    return
  }

  startTime := time.Now()
  octree := BuildOctree(faces, ComputeRootBound(faces), depth)

  var voxelBoxes []BoundingBox
  CollectVoxelBoxes(octree, &voxelBoxes)

  createdNodes := make([]int, depth+1)
  skippedNodes := make([]int, depth+1)
  CollectStats(octree, depth, createdNodes, skippedNodes)

  elapsedTime := time.Since(startTime)

  baseName := strings.TrimSuffix(filepath.Base(filename),
filepath.Ext(filename))
  outPath := filepath.Join("test",
fmt.Sprintf("%s-voxelized-%d.obj", baseName, depth))

  if err := ExportToObj(voxelBoxes, outPath); err != nil {
    fmt.Printf("Error saat menyimpan file output: %v\n", err)
    return
  }

  absPath, _ := filepath.Abs(outPath)
  numVoxels := len(voxelBoxes)

  fmt.Println("\n==================================")
  fmt.Printf("Banyaknya voxel yang terbentuk: %d\n", numVoxels)
  fmt.Printf("Banyaknya vertex yang terbentuk: %d\n",
numVoxels*8)
  fmt.Printf("Banyaknya faces yang terbentuk: %d\n",
numVoxels*12)

  fmt.Println("\nStatistik Octree:")
```

```
  fmt.Println("Kedalaman | Terbentuk | Dilewati")
  fmt.Println("----------------------------------")
  for i := 1; i <= depth; i++ {
    fmt.Printf("Level %-3d | %-9d | %-8d\n", i, createdNodes[i],
skippedNodes[i])
  }
  fmt.Printf("\nKedalaman octree: %d\n", depth)
  fmt.Printf("Lama waktu eksekusi: %v\n", elapsedTime)
  fmt.Printf("Lokasi file disimpan: %s\n", absPath)
  fmt.Println("==================================")
}
```

C. Hasil Program

<table>
<tr><th colspan="2">Percobaan 1</th></tr>
<tr><th>Input</th><th>Output</th></tr>
<tr><td>

```
❯ ./bin/voxelizer.exe examples/cow.obj 8
```

cow.obj

</td><td>

```
Memproses file: examples/cow.obj dengan kedalaman maksimal: 8

==================================
Banyaknya voxel yang terbentuk: 91951
Banyaknya vertex yang terbentuk: 735608
Banyaknya faces yang terbentuk: 1103412

Statistik Octree:
Kedalaman | Terbentuk | Dilewati
-----------------------------------
Level 1   | 8         | 4
Level 2   | 32        | 13
Level 3   | 152       | 65
Level 4   | 696       | 355
Level 5   | 2728      | 1362
Level 6   | 10928     | 5272
Level 7   | 45248     | 22445
Level 8   | 182424    | 90473

Kedalaman octree: 8
Lama waktu eksekusi: 144.6002ms
Lokasi file disimpan: C:\Users\nazwa\Coding\Semester-6\Tucil2_18223003_18223066\test\cow-voxelized-8.obj
==================================
```

```
v -3.793090 -3.637036 -1.701405
v -3.140345 -3.637036 -1.701405
v -3.140345 -2.984291 -1.701405
v -3.793090 -2.984291 -1.701405
v -3.793090 -3.637036 -1.048660
v -3.140345 -3.637036 -1.048660
v -3.140345 -2.984291 -1.048660
v -3.793090 -2.984291 -1.048660
f 1 3 2
f 1 4 3
f 5 6 7
f 5 7 8
f 1 2 6
f 1 6 5
f 4 8 7
f 4 7 3
f 1 5 8
f 1 8 4
f 2 3 7
f 2 7 6
v -3.793090 -2.984291 -1.701405
v -3.140345 -2.984291 -1.701405
v -3.140345 -2.331546 -1.701405
v -3.793090 -2.331546 -1.701405
v -3.793090 -2.984291 -1.048660
v -3.140345 -2.984291 -1.048660
v -3.140345 -2.331546 -1.048660
v -3.793090 -2.331546 -1.048660
f 9 11 10
f 9 12 11
f 13 14 15
```

cow-voxelized-8.obj

</td></tr>
<tr><td>Banyaknya voxel :</td><td>91951</td></tr>
<tr><td>Banyaknya vertex :</td><td>735608</td></tr>
<tr><td>Banyaknya faces :</td><td>1103412</td></tr>
<tr><td>Statistik node octree :</td><td>1 : 8</td></tr>
</table>

	2 : 32 3 : 152 4 : 696 5 : 2728 6 : 10928 7 : 45248 8 : 182424
Statistik node dilewati :	1 : 4 2 : 13 3 : 65 4 : 355 5 : 1362 6 : 5272 7 : 22445 8 : 90473
Kedalaman octree :	8
Lama waktu program :	144.6002 ms
Path file :	Tucil2_18223003_18223066\test\cow-voxelized-8.obj

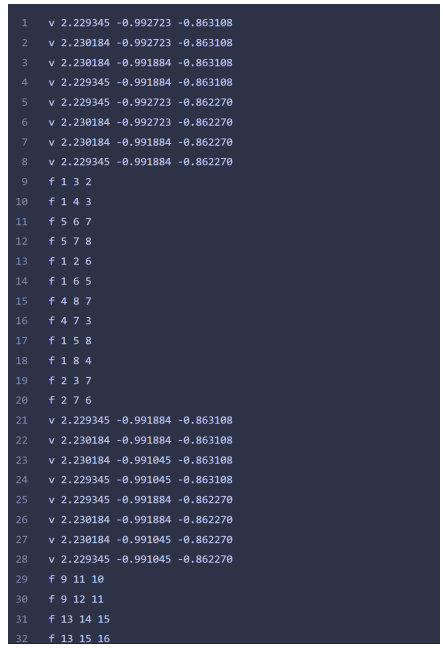
Percobaan 2	
Input	**Output**
`❯ ./bin/voxelizer.exe examples/line.obj 8` line.obj	

```
Memproses file: examples/line.obj dengan kedalaman maksimal: 8

==================================
Banyaknya voxel yang terbentuk: 14787
Banyaknya vertex yang terbentuk: 118296
Banyaknya faces yang terbentuk: 177444

Statistik Octree:
Kedalaman | Terbentuk | Dilewati
----------------------------------
Level 1   | 8         | 5
Level 2   | 24        | 17
Level 3   | 56        | 29
Level 4   | 216       | 132
Level 5   | 672       | 384
Level 6   | 2304      | 1286
Level 7   | 8144      | 4322
Level 8   | 30576     | 15789

Kedalaman octree: 8
Lama waktu eksekusi: 45.2478ms
Lokasi file disimpan: C:\Users\nazwa\Coding\Semester-6\Tucil2_18223003_18223066\t
est\line-voxelized-8.obj
==================================
```

	 line-voxelized-8.obj
Banyaknya voxel :	14787
Banyaknya vertex :	118296
Banyaknya faces :	177444
Statistik node octree :	1 : 8 2 : 24 3 : 56 4 : 216 5 : 672 6 : 2304 7 : 8144 8 : 30576
Statistik node dilewati :	1 : 5 2 : 17 3 : 29 4 : 132 5 : 384 6 : 1286 7 : 4322 8 : 15789
Kedalaman octree :	8
Lama waktu program :	45.2478 ms

```
v 2.229345 -0.992723 -0.863108
v 2.230184 -0.992723 -0.863108
v 2.230184 -0.991884 -0.863108
v 2.229345 -0.991884 -0.863108
v 2.229345 -0.992723 -0.862270
v 2.230184 -0.992723 -0.862270
v 2.230184 -0.991884 -0.862270
v 2.229345 -0.991884 -0.862270
f 1 3 2
f 1 4 3
f 5 6 7
f 5 7 8
f 1 2 6
f 1 6 5
f 4 8 7
f 4 7 3
f 1 5 8
f 1 8 4
f 2 3 7
f 2 7 6
v 2.229345 -0.991884 -0.863108
v 2.230184 -0.991884 -0.863108
v 2.230184 -0.991045 -0.863108
v 2.229345 -0.991045 -0.863108
v 2.229345 -0.991884 -0.862270
v 2.230184 -0.991884 -0.862270
v 2.230184 -0.991045 -0.862270
v 2.229345 -0.991045 -0.862270
f 9 11 10
f 9 12 11
f 13 14 15
f 13 15 16
```

Path file :	Tucil2_18223003_18223066\test\line-voxelized-8.obj

Percobaan 3	
Input	**Output**

```
❯ ./bin/voxelizer.exe examples/pumpkin.obj 6
```

pumpkin.obj

```
Memproses file: examples/pumpkin.obj dengan kedalaman maksimal: 6

==================================
Banyaknya voxel yang terbentuk: 18608
Banyaknya vertex yang terbentuk: 148864
Banyaknya faces yang terbentuk: 223296

Statistik Octree:
Kedalaman | Terbentuk | Dilewati
-----------------------------------
Level 1   | 8         | 0
Level 2   | 64        | 8
Level 3   | 448       | 195
Level 4   | 2024      | 916
Level 5   | 8864      | 4261
Level 6   | 36824     | 18216

Kedalaman octree: 6
Lama waktu eksekusi: 75.1716ms
Lokasi file disimpan: C:\Users\nazwa\Coding\Semester-6\Tucil2_18223003_18223066\test\pumpkin-voxelized-6.obj
==================================
```

```
v -23.646408 -20.289038 -138.032336
v -22.409775 -20.289038 -138.032336
v -22.409775 -19.052406 -138.032336
v -23.646408 -19.052406 -138.032336
v -23.646408 -20.289038 -136.795703
v -22.409775 -20.289038 -136.795703
v -22.409775 -19.052406 -136.795703
v -23.646408 -19.052406 -136.795703
f 1 3 2
f 1 4 3
f 5 6 7
f 5 7 8
f 1 2 6
f 1 6 5
f 4 8 7
f 4 7 3
f 1 5 8
f 1 8 4
f 2 3 7
f 2 7 6
v -24.883040 -19.052406 -138.032336
v -23.646408 -19.052406 -138.032336
v -23.646408 -17.815773 -138.032336
v -24.883040 -17.815773 -138.032336
v -24.883040 -19.052406 -136.795703
v -23.646408 -19.052406 -136.795703
v -23.646408 -17.815773 -136.795703
v -24.883040 -17.815773 -136.795703
f 9 11 10
f 9 12 11
f 13 14 15
```

pumpkin-voxelized-6.obj

Banyaknya voxel :	18608
Banyaknya vertex :	148864
Banyaknya faces :	223296
Statistik node octree :	1 : 8 2 : 64 3 : 448 4 : 2024 5 : 8864 6 : 36824
Statistik node dilewati :	1 : 0 2 : 8 3 : 195 4 : 916 5 : 4261 6 : 18216
Kedalaman octree :	6
Lama waktu program :	75.1716 ms
Path file :	Tucil2_18223003_18223066\test\pumpkin-voxelized-6.obj

Percobaan 4	
Input	**Output**
`❯ ./bin/voxelizer.exe examples/teapot.obj 8` teapot.obj	

```
Memproses file: examples/teapot.obj dengan kedalaman maksimal: 8

================================
Banyaknya voxel yang terbentuk: 118263
Banyaknya vertex yang terbentuk: 946104
Banyaknya faces yang terbentuk: 1419156

Statistik Octree:
Kedalaman | Terbentuk | Dilewati
-----------------------------------
Level 1   | 8         | 4
Level 2   | 32        | 12
Level 3   | 160       | 62
Level 4   | 784       | 354
Level 5   | 3440      | 1656
Level 6   | 14272     | 6972
Level 7   | 58400     | 28881
Level 8   | 236152    | 117889

Kedalaman octree: 8
Lama waktu eksekusi: 162.9795ms
Lokasi file disimpan: C:\Users\nazwa\Coding\Semester-6\Tucil2_18223003_18223066\test\teapot-voxelized-8.obj
================================
```

```
v -39.478509 -18.462642 -41.542452
v -38.764750 -18.462642 -41.542452
v -38.764750 -17.748884 -41.542452
v -39.478509 -17.748884 -41.542452
v -39.478509 -18.462642 -40.828694
v -38.764750 -18.462642 -40.828694
v -38.764750 -17.748884 -40.828694
v -39.478509 -17.748884 -40.828694
f 1 3 2
f 1 4 3
f 5 6 7
f 5 7 8
f 1 2 6
f 1 6 5
f 4 8 7
f 4 7 3
f 1 5 8
f 1 8 4
f 2 3 7
f 2 7 6
v -39.478509 -17.748884 -41.542452
v -38.764750 -17.748884 -41.542452
v -38.764750 -17.035125 -41.542452
v -39.478509 -17.035125 -41.542452
v -39.478509 -17.748884 -40.828694
v -38.764750 -17.748884 -40.828694
v -38.764750 -17.035125 -40.828694
v -39.478509 -17.035125 -40.828694
f 9 11 10
f 9 12 11
f 13 14 15
```

teapot-voxelized-8.obj

Banyaknya voxel :	118263
Banyaknya vertex :	946104
Banyaknya faces :	1419156
Statistik node octree :	1 : 8 2 : 32 3 : 160 4 : 784 5 : 3440 6 : 14272 7 : 58400 8 : 236152
Statistik node dilewati :	1 : 4 2 : 12

	3 : 62 4 : 354 5 : 1656 6 : 6972 7 : 28881 8 : 117889
Kedalaman octree :	8
Lama waktu program :	162.9795 ms
Path file :	Tucil2_18223003_18223066\test\teapot-voxelized-8.obj

Percobaan 5	
Input	**Output**

`❯ ./bin/voxelizer.exe examples/cow.obj 4`
 	
cow.obj
	`Memproses file: examples/cow.obj dengan kedalaman maksimal: 4`
	
	`==========`
	`Banyaknya voxel yang terbentuk: 341`
	`Banyaknya vertex yang terbentuk: 2728`
	`Banyaknya faces yang terbentuk: 4092`
	
	`Statistik Octree:`
	`Kedalaman | Terbentuk | Dilewati`
	`----------`
	`Level 1 | 8 | 4`
	`Level 2 | 32 | 13`
	`Level 3 | 152 | 65`
	`Level 4 | 696 | 355`
	
	`Kedalaman octree: 4`
	`Lama waktu eksekusi: 4.5095ms`
	`Lokasi file disimpan: C:\Users\nazwa\Coding\Semester-6\Tucil2_18223003_18223066\test\cow-voxelized-4.obj`
	`==========`

	v -3.793090 -3.637036 -1.701405 v -3.140345 -3.637036 -1.701405 v -3.140345 -2.984291 -1.701405 v -3.793090 -2.984291 -1.701405 v -3.793090 -3.637036 -1.048660 v -3.140345 -3.637036 -1.048660 v -3.140345 -2.984291 -1.048660 v -3.793090 -2.984291 -1.048660 f 1 3 2 f 1 4 3 f 5 6 7 f 5 7 8 f 1 2 6 f 1 6 5 f 4 8 7 f 4 7 3 f 1 5 8 f 1 8 4 f 2 3 7 f 2 7 6 v -3.793090 -2.984291 -1.701405 v -3.140345 -2.984291 -1.701405 v -3.140345 -2.331546 -1.701405 v -3.793090 -2.331546 -1.701405 v -3.793090 -2.984291 -1.048660 v -3.140345 -2.984291 -1.048660 v -3.140345 -2.331546 -1.048660 v -3.793090 -2.331546 -1.048660 f 9 11 10 f 9 12 11 f 13 14 15 cow-voxelized-4.obj
Banyaknya voxel :	341
Banyaknya vertex :	2728
Banyaknya faces :	4092
Statistik node octree :	1 : 8 2 : 32 3 : 152 4 : 696
Statistik node dilewati :	1 : 4 2 : 13 3 : 65 4 : 355

Kedalaman octree :	4
Lama waktu program :	4.5095 ms
Path file :	Tucil2_18223003_18223066\test\cow-voxelized-4.obj

D. *Link Repository*

Kode program lengkap beserta instruksi penggunaan dan *file executable* dapat diakses melalui *link repository* berikut:

[wisauce/Tucil2_18223003_18223066: Tugas Kecil 2 IF2211 Strategi Algoritma (Semester II tahun 2025/2026)]

E. Surat Pernyataan

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.

Wisa Ahmaduta Dinutama

Nazwan Siddqi Muttaqin

F. Lampiran

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	✅	
2	Program berhasil dijalankan	✅	
3	Hasil konversi .obj terdiri dari kubus-kubus kecil	✅	
4	Program informasi-informasi atau report	✅	
5	Program dapat menerima masukan .obj, dan mengembalikan output	✅	
6	[Bonus] Program menggunakan *concurrency*	✅	
7	[Bonus] Program dapat menampilkan visualisasi .obj		❌